

A Decision-Oriented Conversational Shopping Assistant

Claude does language. Deterministic code does ranking.

Preference Elicitation · Explainable Recommendations · Lifecycle Support

Chenghui Tan

California State University, East Bay · April 2026

Shopping Is a Decision Problem — Not a Search Problem

Why the Existing Model Breaks Down

1 Cognitive Overload

Users juggle every constraint and trade-off themselves.

2 Vague, Evolving Needs

People rarely know exactly what they want when they start.

3 Preferences Get Overwritten

Add a new constraint — earlier choices quietly disappear.

4 Missing Critical Info

Delivery time, promos, availability hidden or absent.

The Decision Burden

Falls on the user.

Decision fatigue compounds with each new option.

Users guess, settle, or abandon the purchase.

Post-purchase regret — trade-offs were never visible.

Trust erodes when recommendations have no rationale.

Existing Platforms Don't Support Decisions

Google Shopping	Amazon Rufus	ChatGPT Shopping	Perplexity	Our System	
Capability	Google Shopping	Amazon Rufus	ChatGPT Shopping	Perplexity	Our System
Preference Elicitation	✗	Partial	Partial	✗	✓
Decision Modeling	✗	✗	✗	✗	✓
Explainable Recommendations	✗	✗	Partial	✓	✓
Delivery / Promo Info	Partial	✗	✗	✗	✓
Preference Continuity	✗	Partial	Partial	✗	✓
Lifecycle Support	✗	✗	✗	✗	✓

Our system targets every gap — in a single decision-oriented flow.

What the Assistant Actually Does

1 Understand You

Situational context

Smart guided Q&A

Continuous flow

e.g. "I need something for cooking"

2 Built for You

Personal AI guide

Preference memory

Multi-criteria model

remembers $\leq \$80$ · 3-day · quiet home

3 Clear Results

Structured — not text heavy

Full attributes visible

Explainable trade-offs

shows the trade-off, not just the rank

+ Lifecycle support — value continues after the purchase.

Five Scenes • One Decision Journey



Each scene corresponds to exactly one architectural layer — UI and architecture are legible to each other.

The Trust Boundary

AI handles language. Deterministic code handles the decision.

Claude (Opus 4.6) — Natural Language

Probabilistic · Translator between messy intent and structured data

① Parse preferences

② Clarifying questions

③ Explain trade-offs

④ Lifecycle suggestions

▼ TRUST BOUNDARY ▼

Rule-Based Recommendation Engine — The Decision

Deterministic · Auditable · Every weight visible · Reproducible across model updates

① Hard constraint filter

② Feasible set

③ Two-tier weighted scoring

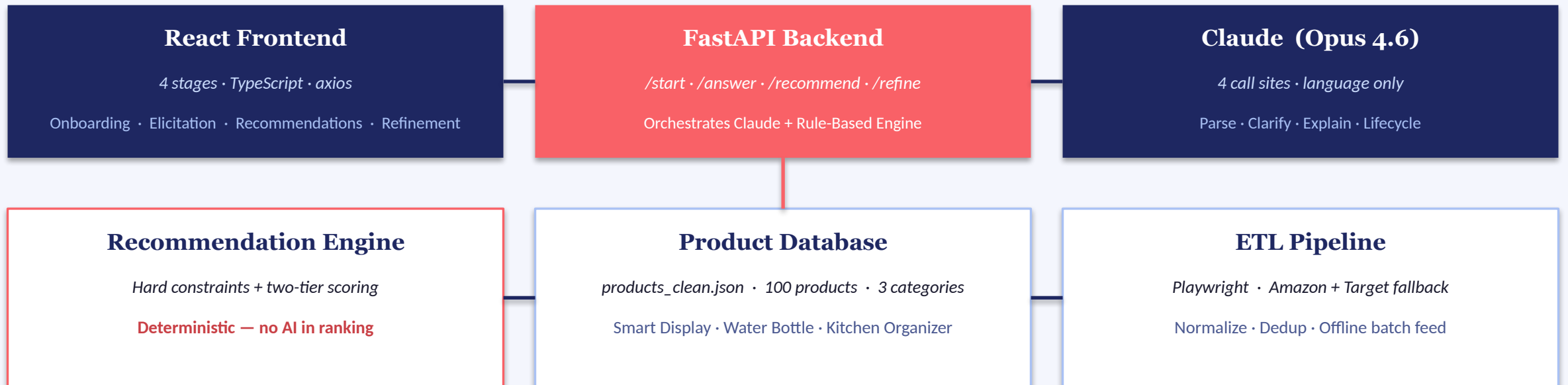
④ Ranked output

Conceptual Layers & Runtime Components

Conceptual Flow



Runtime Components



Deterministic Two-Tier Weighted Scoring

1

Hard Constraint Filter

Budget & delivery deadline — non-negotiable.
Eliminated before scoring.

2

Build Feasible Set

Only products satisfying all hard constraints proceed.
Typically ~100 → 10–20 candidates.

3

Two-Tier Weighted Scoring

General: rating · price · delivery (dynamic weights)
Feature: category-specific attributes

4

Ranked Output

Combined = $\alpha \cdot \text{General} + (1 - \alpha) \cdot \text{Feature}$, $\alpha = 0.6$
Top N returned with all components preserved.

Worked Example — smart display, flexible budget, no rush

GENERAL (dynamic, from user priorities)

Rating 50%

Price 30%

Delivery 20%

FEATURE (category-specific — smart displays)

Voice 40%

Screen 30%

Smart Home 20%

Brand 10%

✓ PASS

Amazon Echo Show 8

Gen 0.82 · Feat 0.88

\$79 · 4.7★ · 2-day

Combined 0.84 → #1

✓ PASS

Google Nest Hub (2nd Gen)

Gen 0.80 · Feat 0.79

\$65 · 4.5★ · 3-day

Combined 0.80 → #2

✗ ELIMINATED

Amazon Echo Show 15

over \$80 budget

\$249 · 4.8★ · 2-day

filtered before scoring

When the system decides what to show the user, AI steps aside.

Playwright ETL • Normalization • Schema



Normalization Examples

"Arrives Mar 14"	→	5 (days from today)
"Get it in 2–3 days"	→	3 (upper bound)
"Ships within a week"	→	7
"1K reviews"	→	1000
"\$79.99"	→	79.99

ScrapedProduct — 10-field TypedDict

flat schema • boring • easy to reason about

• title	• review_count
• url	• delivery_days
• image_url	• source
• price	• category
• rating	• fetched_at

Capability Validation & Scenario Walkthrough

6 / 6

capability gaps
addressed

4

Claude call sites
(none in ranking)

0

LLM inference in
the ranking step

100

products
across 3 categories

Demo — Kitchen Assistant Scenario

- ① **Onboarding** 'I want something for cooking' → smart display
- ② **Elicitation** 5 guided Qs: budget, screen, delivery, household, voice
- ③ **Recommendations** Ranked list + trade-off explanation per product
- ④ **Refinement** Deprioritize screen → re-rank + lifecycle ideas

Validation Findings

- ✓ **Information completeness**
price, rating, delivery, promos — all in one view
- ✓ **Interaction efficiency**
4-stage flow · zero external page navigation
- ✓ **Decision transparency**
every rec carries an explanation tied to priorities
- ✓ **Reproducibility**
identical inputs → identical ranking

Three End-to-End Scenarios — One Per Category

Kitchen Assistant

Sarah • 38 • busy parent

Opening prompt

"I cook every day but my phone screen is too small to follow recipes."

Chip picks

Cooking • Under \$150 • No rush

Top recommendation

Echo Show 8 / Echo Show 5 with Smart Bulb

Lifecycle value

Hands-free recipes • meal-plan dashboard • family hub

Gym Hydration

Mike • 27 • runner

Opening prompt

"My bottle is heavy, leaks, and I can't track my water."

Chip picks

Gym • Insulated • Lightweight

Top recommendation

Owala FreeSip 19oz Insulated (\$14.99)

Lifecycle value

Hydration tracker • workout reminders • flavor ideas

Cabinet Reset

Priya • 31 • small apartment

Opening prompt

"My drawers are chaos. Every morning I waste time hunting."

Chip picks

Cabinets • Not enough space • Stackable

Top recommendation

Brightroom Stackable Pantry Bin Set (\$9–16)

Lifecycle value

Weekly tidy nudges • smart restock • calm counters

Same five-layer architecture • same UI flow • three product categories — generalisability demonstrated.

What This Proves • What It Doesn't • What's Next

Limitations

- ! **Small dataset**
100 products • 3 categories only
- ! **No user study**
heuristic evaluation only
- ! **Static data**
prices & delivery change dynamically
- ! **Rule-based weights**
inferred, not learned

Conclusions & Impact

- ✓ **Decision support feasible**
end-to-end with Claude Opus 4.6
- ✓ **Structured elicitation**
designed to address cognitive load
- ✓ **Explainable trade-offs**
transparent, reproducible rankings
- ✓ **ETL validates the approach**
Playwright scrape • normalize • dedup
- ✓ **Lifecycle value**
extends past the purchase itself

Future Work

- **Formal user study**
SUS + task completion time
- **Live retailer API**
real-time pricing
- **Learned ranking**
from interaction data
- **More categories**
beyond the current three
- **Mobile-first UI**
redesign for small screens

Thank You

Questions & Discussion

Author

Chenghui Tan

Institution

California State University, East Bay · CSUEB

Project

A Decision-Oriented Conversational Shopping Assistant

Stack

React · TypeScript · FastAPI · Python · Claude Opus 4.6 · Playwright